HustleHub Security Test Matrix

1. Summary

Threat area	Scenarios	Passed	Failed	Partial	Blocked	Not run
Injection and input validation	6	4	0	0	0	2
Cross-site scripting (XSS)	2	0	0	1	0	1
Broken authentication	16	5	0	2	0	9
Broken access control	12	3	0	1	0	8
Rate limiting	7	4	0	0	0	3
Misconfiguration	14	9	0	2	0	3
Total	57	25	0	6	0	26

Injection and input validation

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
INJ-01	Invalid input: name T, email not-an-email, password 123	API running. No login needed.	POST /api/auth/register — Postman: Register - Invalid Input	400, input rejected by validation		Not run
INJ-02	NoSQL operator injection: {"email":{"$ne":null},"password":{"$ne":null}} (data is in MongoDB)	API running and connected to MongoDB.	POST /api/auth/login — Manual	400 or 401, no token issued		Not run
INJ-03	Non-text values: array as currentPassword, {"$gt":""} as newPassword	Test dependencies installed. No database needed. (Manual repeat: logged-in user with a valid token.)	POST /api/auth/change-password — Unit: change-password validation	Rejected with must be text, never reaches the database or bcrypt	Passed: 'fields must be text, not arrays or objects'.	Pass
INJ-04	Price input: more than 2 decimals, price sent as text, and a 300-digit number (slow-regex check)	Test dependencies installed. No database needed. (Manual repeat: freelancer logged in.)	POST /api/gigs — Unit: create-gig validation	Decimals and text rejected with clear messages, validation finishes in under 200 ms	Passed: all 4 create-gig tests, including 'the decimals check is fast even on a long input' (5.6 ms).	Pass
INJ-05	Query string abuse: action=drop_database, repeated page, limit=51, unknown parameter actorId	Test dependencies installed. No database needed. (Manual repeat: admin token, audit route path confirmed.)	Audit log query route (confirm path under /api/admin/) — Unit: audit query validation	Each rejected: unknown action, single value only, limit 1 to 50, unknown parameters refused	Passed: 'rejects an unknown action', 'rejects repeated or out-of-range paging values' and 'rejects unknown query parameters'.	Pass
INJ-06	Log injection: CR, LF and other control characters in logged values, and control characters in logged keys	Test dependencies installed. No database needed.	Server logging — Unit: log injection	Control characters become spaces and are removed from keys, so a user cannot forge extra log lines	Passed: 'CR, LF and other control characters become spaces' and 'control characters in keys are removed too'.	Pass

Cross-site scripting (XSS)

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
XSS-01	Script tag in the name field, then view where the name is displayed	Front end and API running. Browser open on the register page.	POST /api/auth/register — Manual	Stored as text or rejected, script never runs		Not run
XSS-02	Script tag in gig title and description, then view the gig in the app	Front end and API running. Freelancer account logged in.	POST /api/gigs — Manual	Output encoded, script never runs	Client tests passed: GigCard renders an escaped malicious title as text with no img element; GigDetail renders an escaped malicious title as plain text with no img element. Description/manual browser check still outstanding.	Partial

Authentication

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
AUTH-01	Valid registration	API running (npm run start:test). Unique email, generated by the collection.	POST /api/auth/register — Postman: Register - Success	201, token returned, passwordHash not in response		Not run
AUTH-02	Valid login	Account registered in AUTH-01.	POST /api/auth/login — Postman: Login - Success	200, token returned		Not run
AUTH-03	Wrong password	Account registered in AUTH-01.	POST /api/auth/login — Postman: Login - Wrong Password	401, message Invalid email or password		Not run
AUTH-04	Unknown email (user enumeration check)	Email address that is not registered.	POST /api/auth/login — Postman: Login - Unknown Email	401, identical message to AUTH-03		Not run
AUTH-05	No token	None. No Authorization header sent.	GET /api/auth/me — Postman: Protected Route - No Token	401		Not run
AUTH-06	Invalid token (Bearer garbage.token.value)	None. Made-up token supplied.	GET /api/auth/me — Postman: Protected Route - Invalid Token	401		Not run
AUTH-07	Valid token	Valid token from AUTH-02.	GET /api/auth/me — Postman: Protected Route - Valid Token	200, returns the caller's own user		Not run
AUTH-08	Account lockout: 5 failed attempts inside 15 minutes. Attempts 1 to 5 may check the password, the 5th sets the lock. While locked, even the correct password is refused	Test dependencies installed. No database needed. API run: freshly registered account, and relaxed limits (npm run start:test) so the per-IP login limiter does not fire first.	POST /api/auth/login — Unit: lock decision. Manual: repeat against the API (see test notes)	Counter goes 1 to 5, lockUntil is 15 minutes after the 5th attempt, no password check while locked	Unit part passed: 'the 5th attempt inside the window locks the account for 15 minutes' and 'while locked, no attempt is claimed'. Start-up log shows the policy is active. API check still to do.	Partial
AUTH-09	Lockout timing: lock expires after 15 minutes and counting restarts, failures older than the window are ignored, slow guessing (one try every 4 minutes) is still caught, a successful login resets the counter	Test dependencies installed. No database needed.	POST /api/auth/login — Unit: lock decision, atomic update	Behaves as described. The atomic update only matches an unlocked account and locks at the limit in the second stage	Passed: all 9 'lock decision' tests and both 'atomic update' tests.	Pass
AUTH-10	Lockout policy cannot be weakened in production: LOCKOUT_MAX_ATTEMPTS=1000 ignored when NODE_ENV=production, zero, text and decimal values ignored	Test dependencies installed. No database needed.	Server configuration — Unit: lockout policy	Policy stays at 5 attempts, 15 minute window, 15 minute lock	Passed: all 4 'lockout policy' tests.	Pass

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
AUTH-11	Input validation: both fields required, text only, unknown field userId rejected, new password at least 8 characters with a number, new differs from current, at most 72 bytes (counted in bytes, so emoji count as 4)	Test dependencies installed. No database needed.	POST /api/auth/change-password — Unit: change-password validation	Each case rejected with the listed message, exactly 72 bytes accepted	Passed: all 7 'change-password validation' tests.	Pass
AUTH-12	Wrong current password with a valid token	Logged-in user who knows their current password.	POST /api/auth/change-password — Manual	400 (not 401), shown as a form error, user stays logged in		Not run
AUTH-13	Password type and length: array as password, password over 72 bytes	Test dependencies installed. No database needed.	POST /api/auth/register, POST /api/auth/login — Unit: register and login validation	Register rejects over 72 bytes and non-text. Login rejects non-text with Password must be text instead of crashing bcrypt	Passed: both 'register and login validation' tests.	Pass
AUTH-14	Log out everywhere: token version bumped, then an old token is reused	Test dependencies installed. No database needed. API run: one account logged in twice, giving two tokens.	POST /api/auth/logout-all — Unit: token version. Manual: log in twice, call logout-all, retry GET /api/auth/me with both old tokens	Old tokens (including the current one) return 401. Tokens issued before versions existed count as version 0	Unit part passed: the 5 token version tests. API check still to do.	Partial
AUTH-15	JWT payload is minimal	Test dependencies installed. No database needed. Test sets its own random JWT_SECRET.	Token issuing — Unit: generate token	Payload keys are only id, role, tv, iat, exp. No email or password hash	Passed: 'a token carries id, role and the token version, nothing else'.	Pass
AUTH-16	Locked account response compared with an unknown email	A locked account (after AUTH-08) and an unregistered email.	POST /api/auth/login — Manual	Response does not make it easy to tell a locked real account from a non-existent one (confirm the intended design with Muaaz)		Not run

Access control

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
ACC-01	No token	None. No Authorization header sent.	GET /api/admin/users — Postman: RBAC > Admin Users - No Token	401		Not run
ACC-02	Client token	Client registered and token saved (RBAC setup requests).	GET /api/admin/users — Postman: RBAC > Admin Users - Client Token	403, generic message You do not have permission to perform this action, no mention of "admin"		Not run
ACC-03	Freelancer token	Freelancer registered and token saved (RBAC setup requests).	GET /api/admin/users — Postman: RBAC > Admin Users - Freelancer Token	403, same generic message		Not run
ACC-04	Register with "role": "admin" (privilege escalation)	None. Unique email, generated by the collection.	POST /api/auth/register — Postman: RBAC > Register - Admin Role	400, success: false, no token issued		Not run
ACC-05	Admin login (needs admin_email / admin_password supplied at run time; skipped otherwise)	Admin account created with npm run seed:admin (reads ADMIN_EMAIL, ADMIN_PASSWORD and ADMIN_NAME from api/.env). admin_email and admin_password supplied to newman at run time.	POST /api/auth/login — Postman: RBAC > Admin Login	200, role is admin		Not run
ACC-06	Admin token	Admin token from ACC-05.	GET /api/admin/users — Postman: RBAC > Admin Users - Admin Token	200 (confirm against the collection, the request was cut off)		Not run
ACC-07	Append-only: updateOne, updateMany, findOneAndUpdate, findByIdAndUpdate, replaceOne, findOneAndReplace, deleteOne, deleteMany, findOneAndDelete, findByIdAndDelete, bulkWrite, re-saving a loaded entry, and deleting or updating a loaded entry	Test dependencies installed. No database needed.	Audit log (database model) — Unit: audit log model	Every one refused with an append-only error	Passed: all 13 'append-only guards' tests.	Pass

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
ACC-08	Entry content is restricted: only known actions, unknown top-level fields (email) refused, unknown meta fields (password) refused, whitelisted meta (failedAttempts, lockMinutes) allowed	Test dependencies installed. No database needed.	Audit log (database model) — Unit: audit log model	Refused or accepted as described. Newest-first indexes exist	Passed: all 5 'what an entry may contain' tests.	Pass
ACC-09	Actor and IP come from the verified request, not the client. An explicit actor is used only when nobody is logged in (lockout)	Test dependencies installed. No database needed.	Audit log (recording) — Unit: record audit	Stored actorId, actorRole, ip match the request user, lockout entry uses the supplied actor	Passed: 'writes the actor and ip from the verified request' and 'uses an explicit actor when nobody is logged in (lockout)'.	Pass
ACC-10	Who can read the audit log	Client, freelancer and admin tokens available. Audit route path confirmed.	Audit log route (confirm path under /api/admin/) — Manual	No token 401, client 403, freelancer 403, admin 200		Not run
ACC-11	Send a body such as {"userId": "<another user's id>"}	Test dependencies installed. No database needed. API run: two accounts, both with active tokens.	POST /api/auth/logout-all — Unit: logout-all validation. Manual: confirm the other user's token still works	Any body field rejected with Request contains fields that are not allowed, only the caller's own sessions end	Unit part passed: both 'logout-all validation' tests. API check still to do.	Partial
ACC-12	Admin seeding is the only way to create an admin: a password under 12 characters or an invalid email is refused, an existing email is skipped without changes, the password and database URI are never printed	Throwaway database reachable. api/.env has ADMIN_EMAIL, ADMIN_PASSWORD and ADMIN_NAME.	Command line — Manual: npm run seed:admin	Bad password or email refused with nothing created. A second run with the same email is skipped, role unchanged. Output shows no password or database URI		Not run

Rate limiting

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
RATE-01	5 failed logins per 15 minutes per IP, then a 6th attempt	Strict limits (npm run start:test:strict). API restarted so the window is clean.	POST /api/auth/login — Postman: Rate Limits > Login Limiter	Attempts 1 to 5 return 401. Attempt 6 returns 429 with Retry-After between 1 and 900, message Too many requests, please try again in N seconds. where N matches Retry-After, standard RateLimit header present, no X-RateLimit-* headers	Passed (newman, 2026-11-09): attempts 1 to 5 returned 401, attempt 6 returned 429 in 11 ms. All 4 checks passed: Retry-After in range, message matches Retry-After, standard RateLimit headers present, no X-RateLimit-* headers.	Pass
RATE-02	10 bookings per 10 minutes per user, then an 11th	Strict limits. API restarted. Client, freelancer and gig created by the collection setup requests.	POST /api/bookings — Postman: Rate Limits > Booking Limiter	Bookings 1 to 10 return 201. Booking 11 returns 429 with Retry-After between 1 and 600 and the same message format	Passed (newman, 2026-11-09): setup requests (client, freelancer, gig) all returned 201, bookings 1 to 10 returned 201, booking 11 returned 429 in 96 ms. All 4 checks passed.	Pass
RATE-03	Limits cannot be raised in production: RATE_LIMIT_LOGIN_MAX=500 honoured in development, ignored in production, invalid values (abc, -3) ignored	Test dependencies installed. No database needed.	Server configuration — Unit: rate limit config	Production keeps strict limits and reports the ignored override	Passed: all 4 rate limit config tests.	Pass

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
RATE-04	Behind a proxy: with TRUST_PROXY=1, two forwarded IPs get separate buckets. Unset, everyone shares one bucket and a warning is logged. TRUST_PROXY values true, loopback, 10.0.0.0/8, -1, 1.5, 99 rejected. The real app logs no rate limiter warnings	Test dependencies installed. The test starts its own local server.	Any rate-limited route — Unit: trust proxy	Behaves as described, so a client cannot dodge or share a limit by spoofing X-Forwarded-For	Passed: all 3 'resolveTrustProxy' tests, both 'rate limit buckets behind a proxy' tests, and 'the real app with TRUST_PROXY=1 logs no rate limiter validation warnings'.	Pass
RATE-05	Register limiter: 10 registrations per hour per IP, then an 11th (limit taken from the start-up log)	Strict limits. API restarted so the window is clean.	POST /api/auth/register — Manual or Postman runner (not in the Rate Limits collection seen so far, confirm with Virona)	Registrations 1 to 10 return 201, the 11th returns 429 with Retry-After		Not run
RATE-06	Password change limiter: the start-up log shows its limit as [REDACTED], so the value is unknown. Confirm it with Muaaz, then repeat change attempts until it triggers	Strict limits. API restarted. Logged-in user. Limit value confirmed.	POST /api/auth/change-password — Manual	429 with Retry-After once the limit is reached (value to confirm)		Not run
RATE-07	General limiter: 100 requests per 15 minutes per IP on a normal API route (health check excluded)	Strict limits. API restarted.	Any non-health route such as GET /api/auth/me — Postman runner	Request 101 returns 429. Other routes carry a RateLimit-Policy header		Not run

Misconfiguration

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
MISC-01	Health check, and it is not rate limited (110 calls all return 200, no RateLimit header)	API running.	GET /api/health — Postman: Health Check. Unit: health	200, body is exactly {"success":true,"data":{"status":"ok"}}. Other routes still carry RateLimit-Policy	Unit part passed: 'GET /api/health says ok and nothing else' and 'the health check is not rate limited'. Postman request still to run.	Partial
MISC-02	Duplicate email, check error handling	Email already registered by Register - Success earlier in the run.	POST /api/auth/register — Postman: Register - Duplicate Email	409, no stack trace or file paths in the response		Not run
MISC-03	Request from a disallowed origin	Front end and API running. Request sent from an origin that is not allowed.	Any endpoint — Manual	CORS blocks it		Not run
MISC-04	Security headers. Health route covered by unit test, check a few other routes by hand in dev tools	API running. Browser dev tools open.	Any endpoint — Unit: health. Manual (browser dev tools)	Content-Security-Policy containing default-src 'self', X-Content-Type-Options: nosniff, no X-Powered-By	Unit part passed: 'the health check still gets the security headers'. Other routes still to check by hand.	Partial
MISC-05	Environment validation: missing or blank JWT_SECRET, missing MONGO_URI, JWT_SECRET under 32 characters in production only	Test dependencies installed. No database needed. Test passes in its own environment values.	Server start-up — Unit: env validation	Start-up refuses with a clear message, and no message contains a secret value	Passed: all 5 env validation tests.	Pass
MISC-06	Fields never exposed: passwordHash, failedLoginAttempts, firstFailedLoginAt, lastFailedLoginAt, lockUntil, tokenVersion, _id, __v	Test dependencies installed. No database needed.	Any response containing a user — Unit: user model	None present. id, name, role are. New accounts start with 0 failures and token version 0	Passed: both user model tests.	Pass
MISC-07	Audit write fails (database down)	Test dependencies installed. No database needed.	Audit log (recording) — Unit: record audit	Returns false and never throws, logs an EVENT:ERROR line with the action and error name, and no connection string or secret	Passed: 'a failed write is logged as an error event and never throws'.	Pass

ID	Scenario	Preconditions	Request / action	Expected result	Actual result	Status
MISC-08	Secrets are redacted from logs: values under sensitive keys (any case, deep in objects and arrays), JWTs, Axiom tokens, Bearer values, connection-string credentials (host kept), emails masked	Test dependencies installed. No database needed.	Server logging — Unit: sensitive keys, secrets inside strings, record shape	No token, password or credential appears in a log line	Passed: all 'sensitive keys' and 'secrets inside strings' tests, and 'records are scrubbed before they are printed'.	Pass
MISC-09	Safe logging of errors and large values: an Error logs name and message only, long strings cut, depth capped, wide objects capped, circular references marked	Test dependencies installed. No database needed.	Server logging — Unit: special values, size and shape limits	No stack traces or file paths in logs, and a logging failure never throws into the caller	Passed: all 'special values' and 'size and shape limits' tests, and 'a logging failure never throws into the caller'.	Pass
MISC-10	Log shipping setup: off when not configured, needs an https URL and token, plain http only to loopback in test runs, token sent only in the Authorization header, redaction applied before sending	Test dependencies installed. No database needed.	Log shipping — Unit: configuration, sending	Secrets never leave in log records, and the reason shown for a problem never contains the URL or token	Passed: all 5 'configuration' tests, 'sends the token in the Authorization header' and 'redaction is applied before sending'.	Pass
MISC-11	Log shipping failures stay contained: problems reported at most once a minute without URL or token, a throwing fetch never throws into the caller, shipper failures are not shipped again, shutdown gives up at its deadline	Test dependencies installed. No database needed.	Log shipping — Unit: failures stay contained	A failing log service cannot crash the API, leak the token or cause a log loop	Passed: all 3 'failures stay contained' tests and 'close() gives up at its deadline when the service hangs'.	Pass
MISC-12	Request ID cannot be set by the client: every response gets a fresh server-generated X-Request-Id, a client-supplied one is ignored	Test dependencies installed. The test starts its own local server.	Any route — Unit: request id	A client cannot choose or reuse a request ID to confuse or forge log trails	Passed: 'every response gets a fresh server-generated X-Request-Id' and 'a client-supplied X-Request-Id is ignored'.	Pass
MISC-13	Access log privacy: one line per request with the path but not the query string, 5xx logged as error, health check and CORS preflights not logged	Test dependencies installed. The test starts its own local server.	Any route — Unit: access logging	Query strings (which can carry tokens) never reach the logs	Passed: 'one access line per request: method, path without the query string, status, duration' and the related tests.	Pass
MISC-14	npm run start:test refuses to run in production (relaxed limits can never reach production)	No server already running on the port.	Command line — Manual: start with NODE_ENV=production set, then run npm run start:test	Prints start:test refuses to run when NODE_ENV is production, exits with code 1, server does not start		Not run

Client-side automated security regression checks

Area	Automated coverage shown in run	Result	Security-test interpretation
XSS output handling	GigCard and GigDetail render escaped malicious titles as text; no img element is rendered.	Pass (client tests)	Supports XSS-02 title-rendering coverage only. Description and full browser scenario remain manual.
Client-side access control	ProtectedRoute redirects logged-out visitors, denies the wrong role, allows the correct role, and waits during session checks.	Pass (client tests)	Verifies route/UI gating; does not prove API endpoint authorization.
Admin data exposure	AdminUsers renders users without a password field.	Pass (client test)	Client display check only; server-side response-field protection remains covered separately.
Authentication and session handling	Login/register generic errors; register offers only client/freelancer roles; API client attaches Bearer tokens, omits Authorization without a token, and clears session on eligible 401 responses.	Pass (client tests)	Regression coverage for client auth behaviour; does not replace direct API token and role tests.

Area	Automated coverage shown in run	Result	Security-test interpretation
Input and request handling	Client validation blocks invalid gig/register/password input; API client rejects absolute URLs and handles network/non-JSON/API errors and 429 responses.	Pass (client tests)	Supports client-side validation and safe request handling.
Security headers / CSP	Tests check shared security-header values, preview responses, generated Nginx headers, and Nginx proxy certificate/forwarding configuration.	Pass (client tests)	Configuration regression checks; manual deployed-header verification remains necessary.